# How the Photo Booth Works

Follow your photo from the click to your phone

Big writing: for everyone </> Small print: for big nerds

## Your photo's journey

1 — photo: tablet.jpg — booth tablet on its post

**Click!** You tap the moon. A tablet with a tiny computer inside takes your picture.

</> Guition 10.1" touchscreen, **ESP32-P4** chip (2 cores, 400 MHz, 32 MB), 2-megapixel OV02C10 camera, ESP32-C6 for Wi-Fi. Screens drawn with LVGL 9.

2 — photo: light.jpg — Key Light above the tablet

**Lights!** The tablet's camera struggles in the dark, so a light glows for 3·2·1 and switches off at the click.

</> An **Elgato Key Light Mini** controlled over Wi-Fi. Brightness follows how hard the camera sensor is working.

3 — photo: server.jpg — the home server

**Whoosh!** Your photo flies over Wi-Fi to a computer in the house that does the heavy lifting.

</> HTTP upload to a **FastAPI** (Python) app in **Docker** on "pier", an Ubuntu virtual machine on a VMware ESXi server.

4 — photo: cameras.jpg — the security cameras

**Who else was here?** A security camera snaps the driveway, where everyone gathers, and a robot brain counts everyone who's come by tonight.

</> Reolink 4K snapshot. The counter runs **YOLO11n** + ByteTrack on a 4K stream at 4.5 frames/s, on "wharf", a more powerful PC, in Docker.

5 — photo: sample.jpg — a finished booth photo

**Glue it together** The server adds the spooky frame, the driveway snapshot and the visitor count, then files it away.

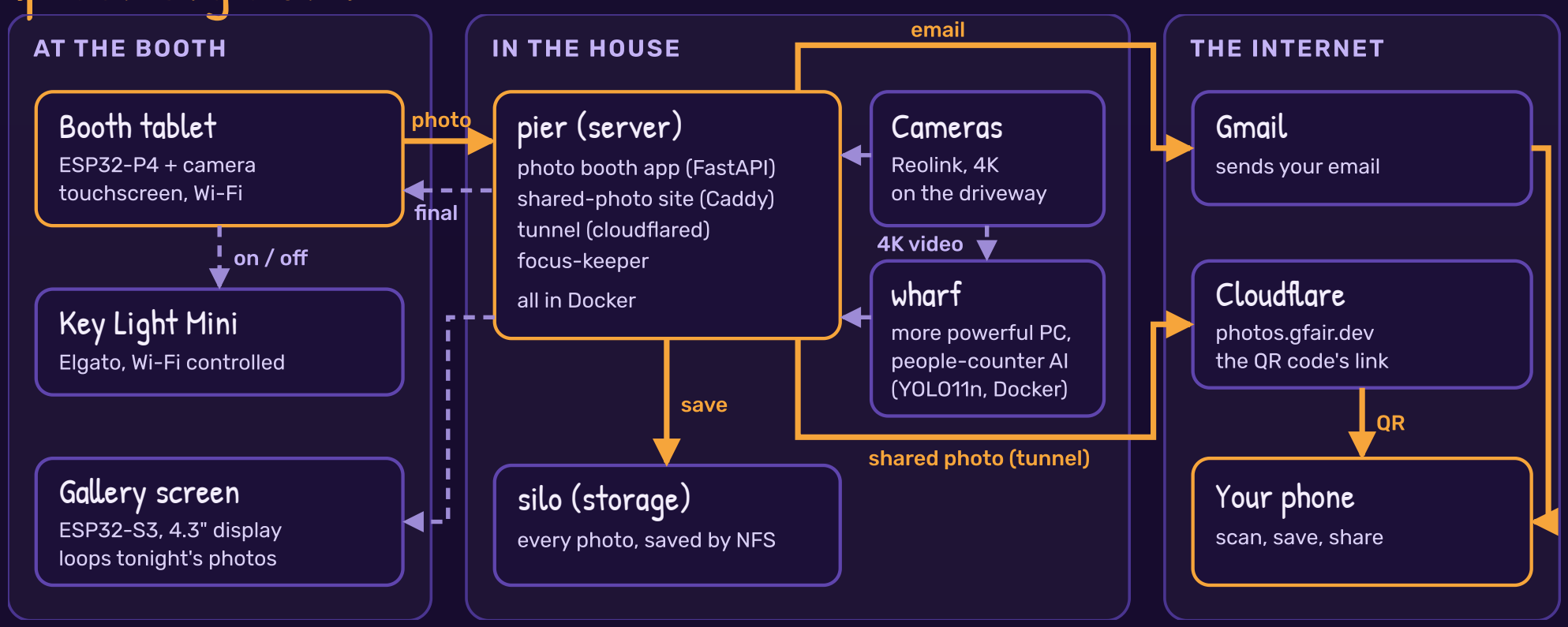
</> Python's **Pillow** layers frame, tilted snapshot and caption into final.jpg, saved over NFS to "silo", a storage server.

6 — photo: gallery.jpg — the little gallery screen

**Take it home** Scan the QR code or get an email. The little gallery screen keeps showing everyone's photos.

</> QR links go to **photos.gfair.dev** through a Cloudflare Tunnel: no open router ports, random 32-letter names. Gallery: ESP32-S3, 4.3" screen.

## Map for big nerds




Want the whole story? Scan for diagrams, parts and the story of the night.

The code isn't public, but we're happy to share it with friends. Just ask!

# Why It's Built This Way

Small chips, reused gear and a few clever tricks

## The right tiny computer

The booth's brain is an ESP32, the kind of chip inside smart plugs. It cost under a tenth of an iPad, and it suits a booth better:

- Turns on straight into the booth
- No other apps for guests to wander into
- Camera, touchscreen and Wi-Fi on one board, made for gadgets like this

**2** cores at 400 MHz

**32** MB of memory

An iPad: several cores at about 3 GHz and 6,000+ MB of memory, roughly **200× more**. So every screen and camera setting here was written from scratch.

## The light: reused, and needed

The tablet's camera is poor in low light, and the booth runs after dark. We already had a Key Light, and it can be switched over Wi-Fi, so the booth turns it on for 3·2·1 and off at the click.

</> Elgato Key Light Mini, controlled over HTTP. Brightness follows the camera sensor's gain.

Also reused: the security cameras, the home servers and the 3D printer (it made the tablet frame, post cap, gallery stand and battery holder).

## Three ways to run a program

### Just a program

Runs straight on the computer, like an app on a phone.

**Here:** the booth tablet and gallery screen each run one program, right on the chip.

### Containers

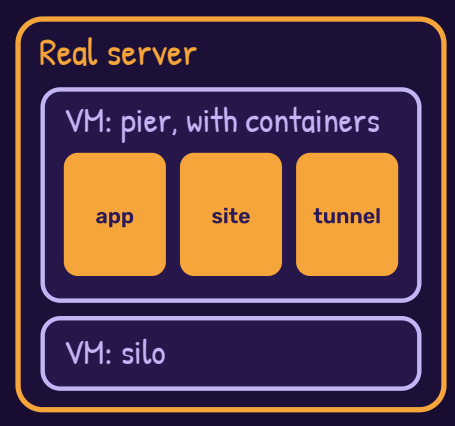
A lunchbox: a program packed with everything it needs, so it runs the same anywhere. Lots of them share one computer.

**Here:** the photo booth app, the shared-photo site, the tunnel, focus-keeper and the people counter all run in Docker.

### Virtual machines

A pretend computer inside a real one, with its own operating system. One real server can act like many computers.

**Here:** "pier" and "silo" are virtual machines on one VMware ESXi server.




## What went wrong (and how we fixed it)

### Ghost photos

The booth took pictures by itself.

A timeout was mistaken for a tap. Now it waits for a real one.

### Too bright

Faces glowed white in the dark yard.

Retuned exposure for one lit face on a black night.

### Blurry camera

The counter camera focused on random stuff.

"focus-keeper" puts the focus back every minute.

### Slow counting

The AI couldn't keep up.

Moved to a more powerful PC: sharper video, 4.5 frames/s.

### Dead battery

It died two days before Halloween.

New battery, new 3D-printed holder the same day.

## Keeping your photo private

- No doors open on our router.
- Shared links are random and secret.
- The booth doesn't save your email address.
- The counter counts people, not faces.

## Built with free software

ESP-IDF · FreeRTOS · LVGL · esp_video · Python · FastAPI · Pillow · Docker · Ubuntu · Caddy · Cloudflare Tunnel · VMware ESXi · Ultralytics YOLO11 · ByteTrack · Patrick Hand font · build123d

</> **Claude**, an AI, wrote **7,075 lines of code** to make this Halloween special: 18 per trick-or-treater!

Want to build one? ESP32 boards start at about $5 and the tools are free. Make a light blink, then keep going. Scan for the docs; ask us for the code.